

Structural Operational Semantics for String Diagrammatic Languages

Fabio Zanasi



Based on joint work with

Filippo Bonchi

Robin Piedeleu

Pawel Sobocinski

String Diagrams: a Syntax for Graphical Models

Enter String Diagrams

String Diagrams originated in **category theory** as a notation for morphisms of **monoidal categories**

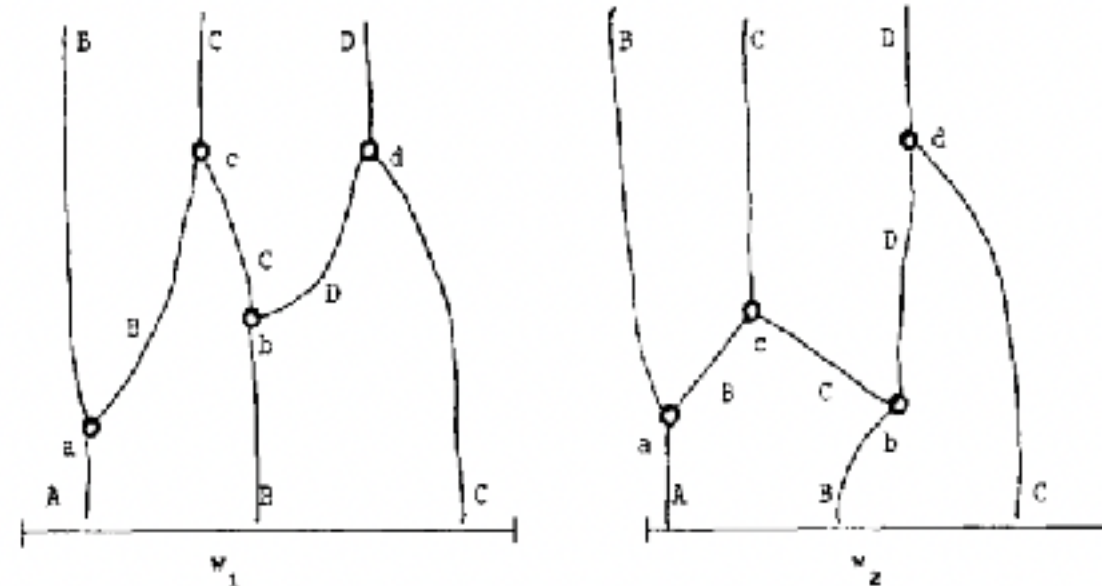
ADVANCES IN MATHEMATICS 88, 55-112 (1991)

JOYAL AND STREET

The Geometry of Tensor Calculus, I

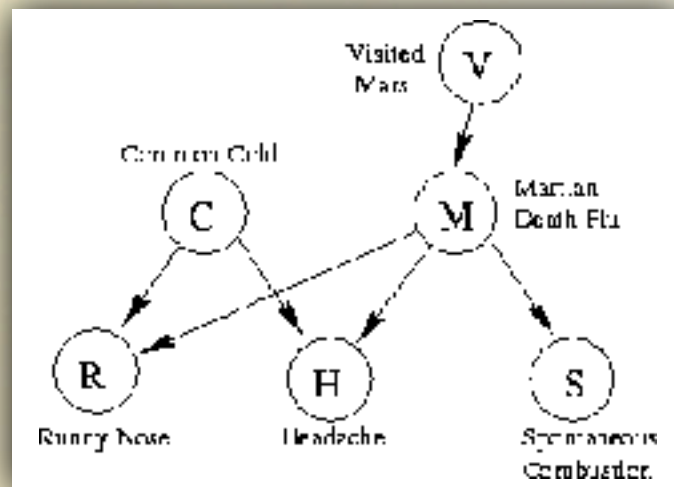
$$(B \otimes c \otimes d) \circ (B \otimes B \otimes b \otimes C) \circ (a \otimes B \otimes C) = w_1$$

$$(B \otimes C \otimes d) \circ (B \otimes c \otimes D \otimes C) \circ (a \otimes b \otimes C) = w_2$$

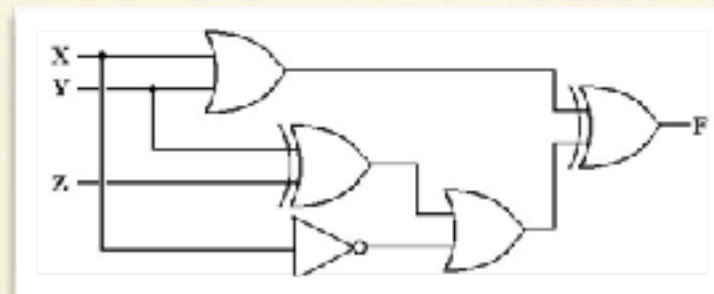


Graphical Models of Computation

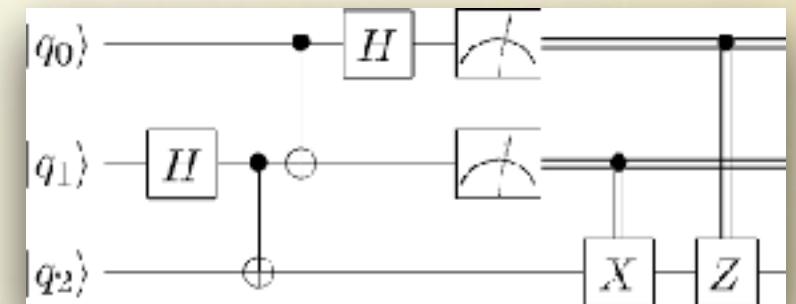
Bayesian networks



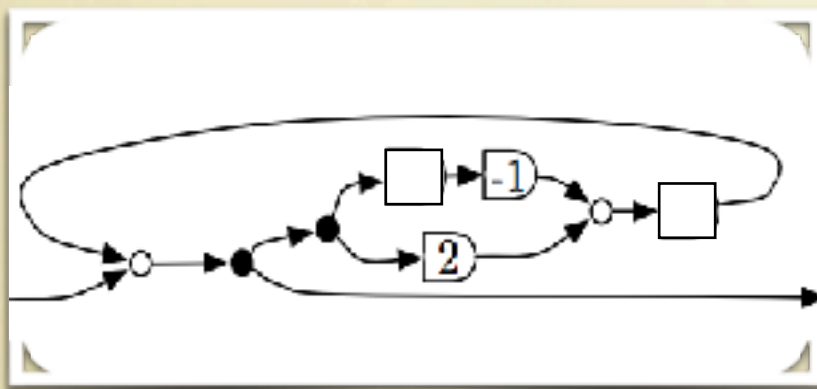
Digital Circuits



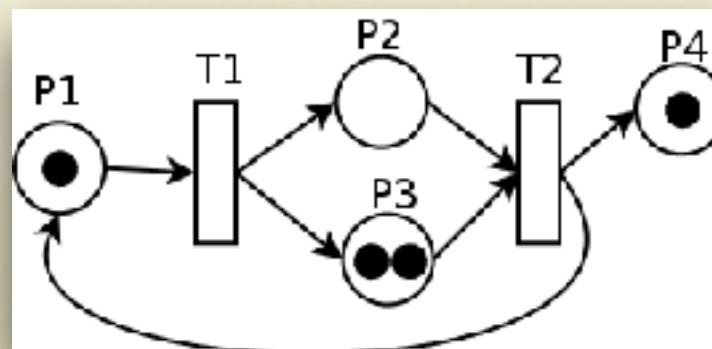
Quantum circuits



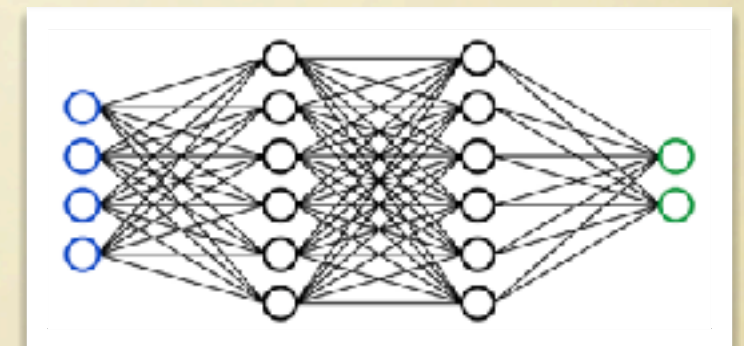
Signal flow graphs



Petri nets

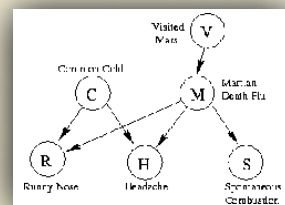
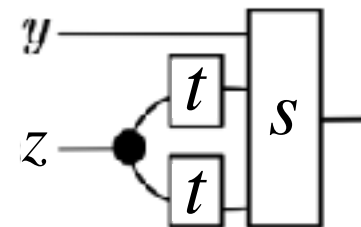
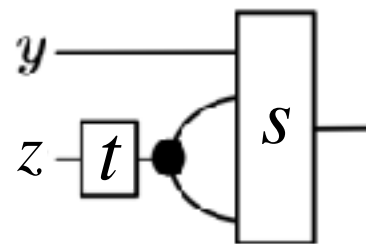


Neural Networks



Why String Diagrams and not Terms?

A graphical representation can tell the difference.



$\longrightarrow s(y, t(z), t(z))$

Is resource z **consumed** once or twice?

```
z <- Bernoulli(0.5)
return (z,z)
```

VS

```
return (Bernoulli(0.5), Bernoulli(0.5))
```

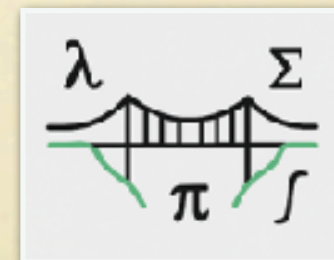
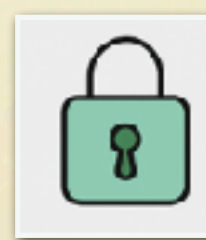
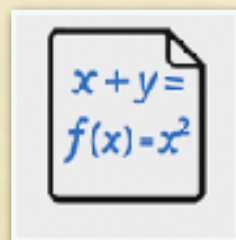
A Modern Take on String Diagrams

String Diagrams serve as a **syntax**
for graphical models.

In the same sense as logic/programming languages/process algebra

$$\begin{array}{lcl} b & ::= & \text{True} \mid x = y \mid x = n \mid \neg b \mid b \wedge b \mid b \vee b \\ p & ::= & \text{skip} \mid x := y \mid x := s(y) \mid x := n \mid \text{while } b \text{ do } p \mid p ; p \end{array}$$

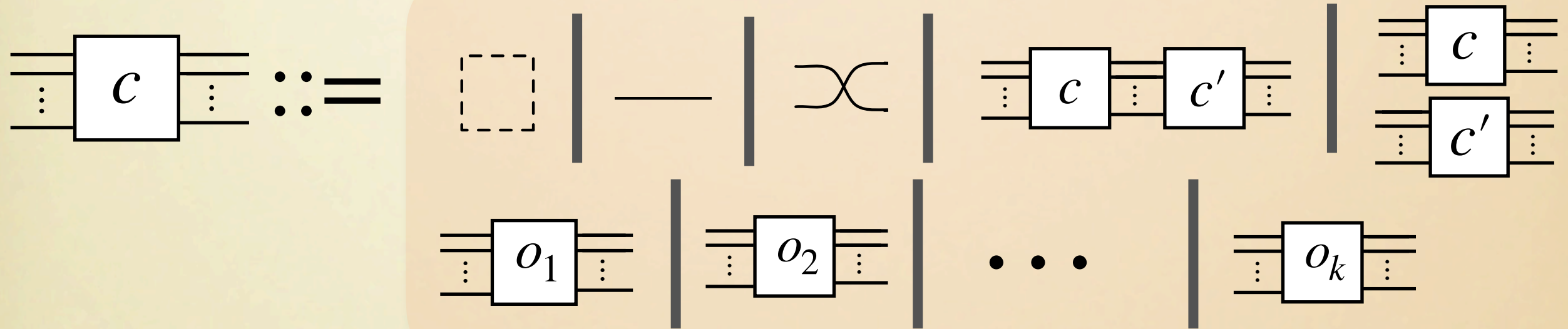
String diagrammatic syntax enables
algebraic semantics for graphical models.



String Diagrammatic Syntax

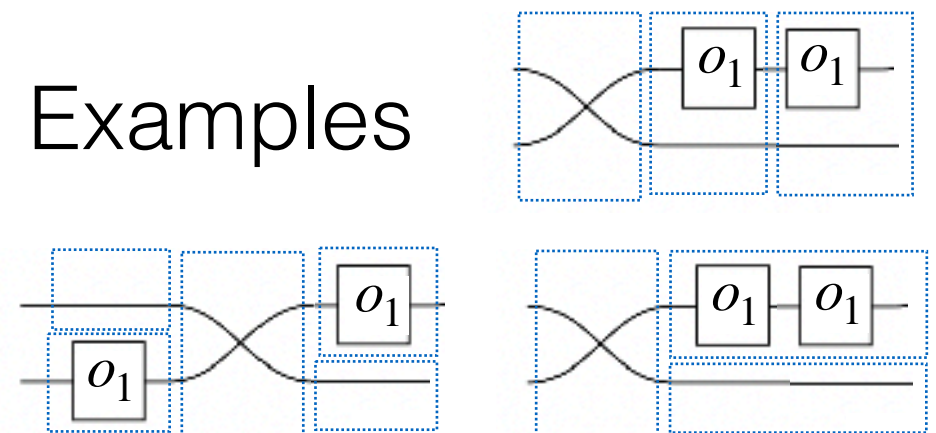
basic wiring

two forms of composition



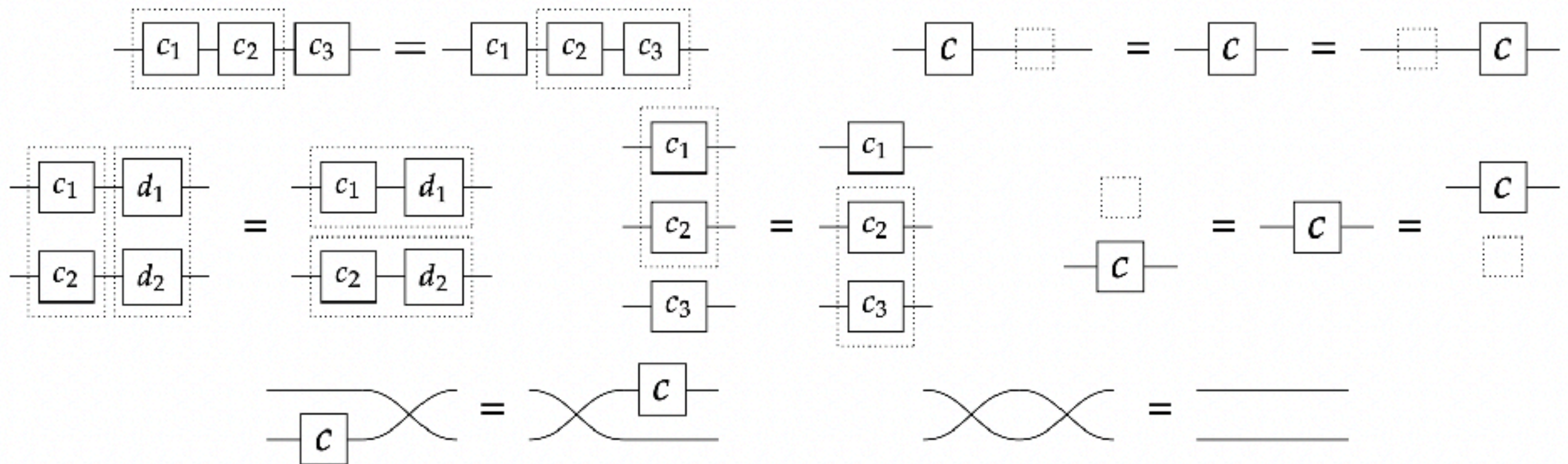
domain specific operations

Examples



String Diagrammatic Syntax

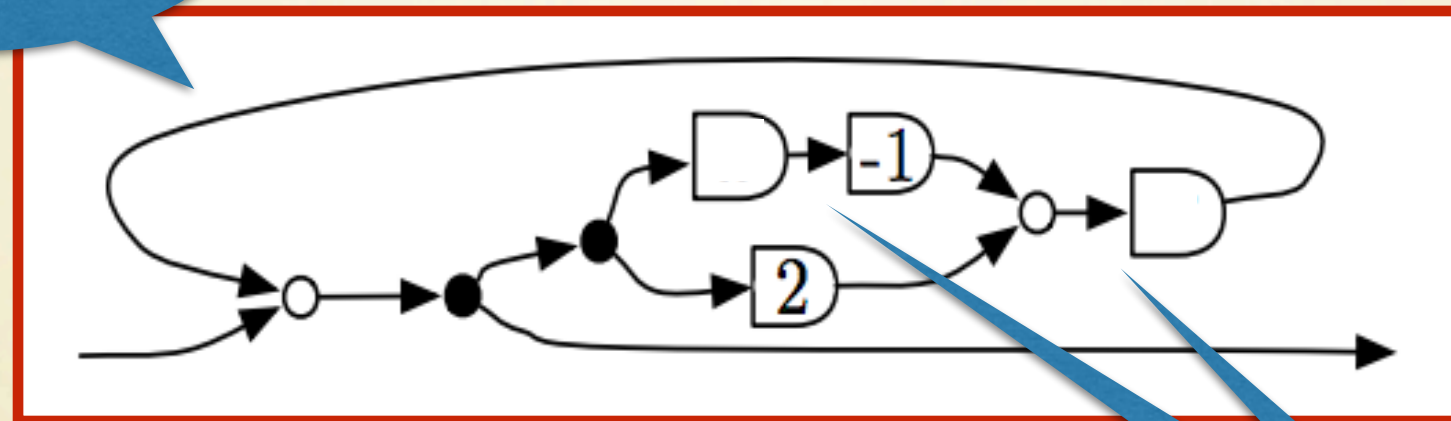
String diagrammatic syntax is defined modulo the laws of **symmetric monoidal categories**:



These equations allow us to reason formally about string diagrams interchangeably as **syntax** and as **graphs**.

Example: Signal Flow Graphs

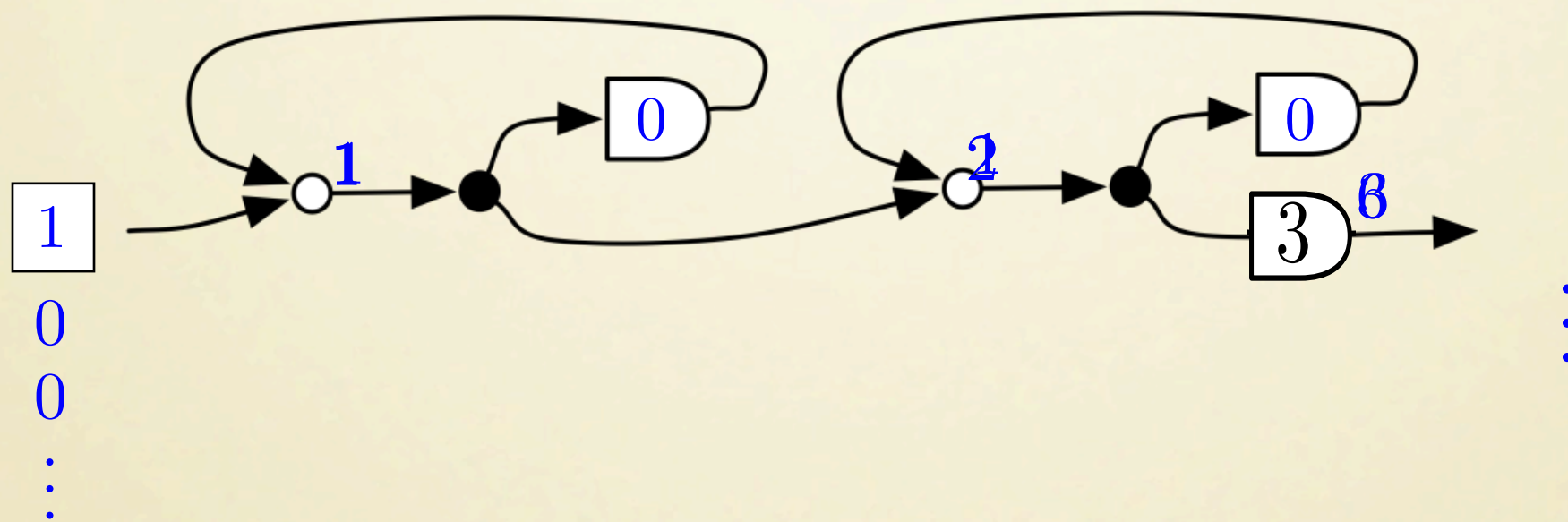
feedback
loop



Signal Flow Graphs
(Shannon 1942)

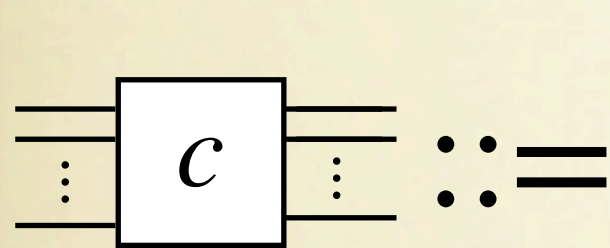
registers

Executing a Signal Flow Graph



$$(c,0,0) \xrightarrow{\frac{1}{3}} (c,1,1) \xrightarrow{\frac{0}{6}} (c,1,2) \xrightarrow{\frac{0}{9}} (c,1,3) \xrightarrow{\frac{0}{12}} \dots$$

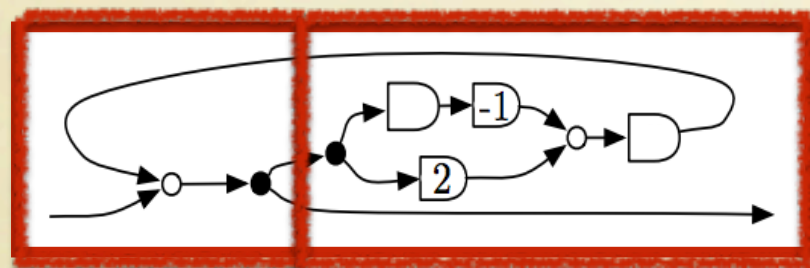
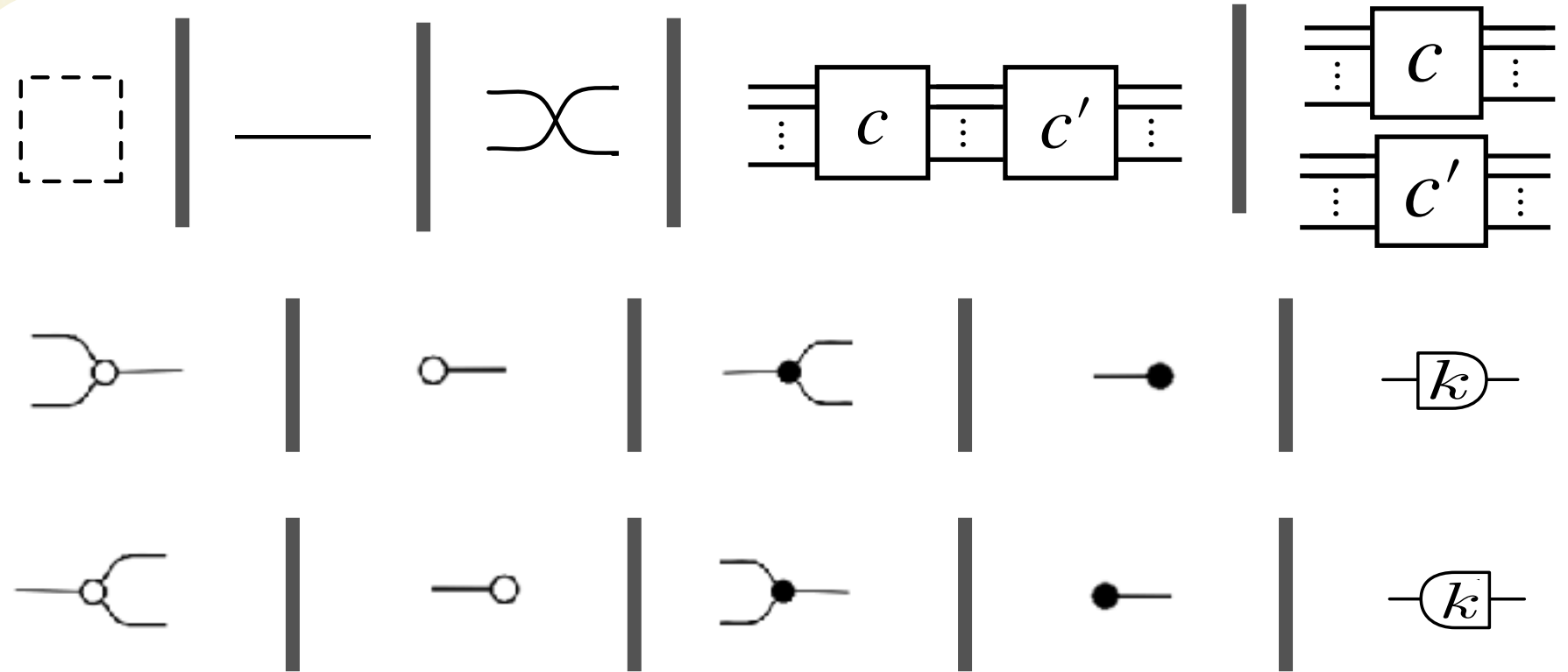
Signal Flow Diagrams



$k \in R$

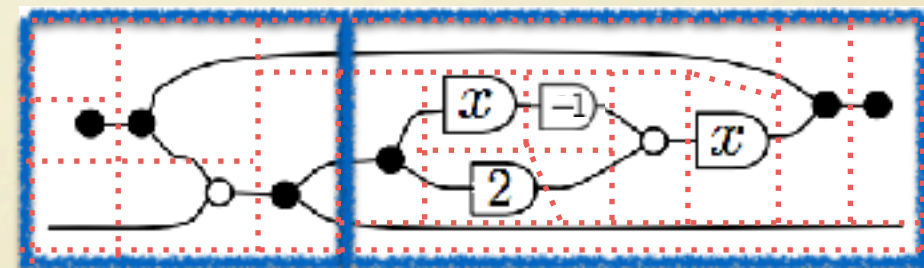
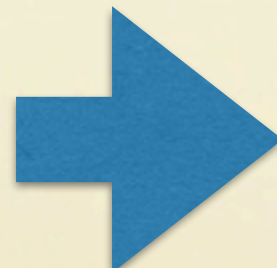
In this case,

$R = \mathbb{R}[x]$



$\updownarrow$
?

$\updownarrow$
?



$\updownarrow$
 R_1

;

$\updownarrow$
 R_2

=

R

Signal Flow Diagrams: Semantics

The **denotational semantics** of a signal flow diagram is a **relation over R-vectors**

$$\langle \text{Addition} \rangle_R := \{((\begin{smallmatrix} x \\ y \end{smallmatrix}), x + y) \mid x, y \in R\}$$

Addition

$$\langle \text{Zero} \rangle_R := \{(\bullet, 0)\}$$

Zero

$$\langle \text{Duplicate} \rangle_R := \{(x, (\begin{smallmatrix} x \\ x \end{smallmatrix})) \mid x \in R\}$$

Duplicate

$$\langle \text{Discard} \rangle_R := \{(x, \bullet) \mid x \in R\}$$

Discard

$$\langle \text{Multiply by } k \rangle_R := \{(x, k \cdot x) \mid x \in R\}$$

Multiply by k

$\vdots$

$$\langle \text{Synch} \rangle_R := \{(a, b) \mid \exists w. (a, w) \in \langle c \rangle_R, (w, b) \in \langle d \rangle_R\}$$

Synch

$$\langle \text{Parallel} \rangle_R := \{((\begin{smallmatrix} a_1 \\ a_2 \end{smallmatrix}), (\begin{smallmatrix} b_1 \\ b_2 \end{smallmatrix})) \mid (a_1, a_2) \in \langle c \rangle_R, (b_1, b_2) \in \langle d \rangle_R\}$$

Parallel

Signal Flow Diagrams: Results

Denotational semantics

Complete axiomatisation

Other models and extensions:

Petri nets

Electrical circuits

Gaussian Probability

Linear Regression

What about operational semantics?

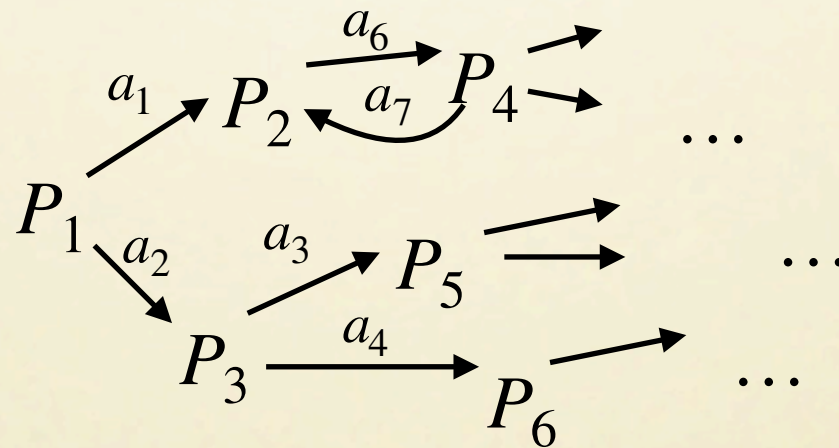
A Brief Recap of Structural Operational Semantics

Operational Semantics

- Meaning of a process = the actions performed during its execution.

$$P \xrightarrow{a} Q$$

- Formally given by a **labeled transition system**.



- Two processes are **bisimilar** if every action one can perform, the other can match, and the resulting processes are again bisimilar.

Structural Operational Semantics

- Defined by clauses following the **syntactic structure** of terms in a language.
- Example. Communication rule in CCS:

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau} P'|Q'}$$

- **Compositionality:**
 - the semantics of a process is determined by the semantics of its parts.
 - replacing a sub-process with a bisimilar one does not change the behaviour of the whole process.

Categorical Interpretation of SOS

Compositional
SOS rule specifications

$$\begin{array}{c}
 \frac{}{\mu.P \xrightarrow{\mu} P} \quad \frac{P \xrightarrow{\mu} P'}{P+Q \xrightarrow{\mu} P'} \\
 \\
 \frac{P \xrightarrow{\mu} P'}{P\parallel Q \xrightarrow{\mu} P'\parallel Q} \\
 \\
 \frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P\parallel Q \xrightarrow{\tau} P'\parallel Q'}
 \end{array}$$

$\sim$

Distributive laws of
structure over behaviour

$$\begin{array}{l}
 \lambda : \mathcal{TF} \rightarrow \mathcal{FT} \\
 \lambda : \mathcal{TP}(A \times -) \rightarrow \mathcal{P}(A \times \mathcal{T}(-))
 \end{array}$$

[Turi and Plotkin, 1997]

$\mathcal{TX}$ terms

$\mathcal{TX} \xrightarrow{\beta} \mathcal{FTX}$ SOS rules

Compositionality, Categorically

Unique coalgebra map: assigns to each term its LTS (modulo bisimilarity)

$\mathcal{T}$ -algebra:
term syntax

$$\frac{}{\mu.P \xrightarrow{\mu} P} \quad \frac{P \xrightarrow{\mu} P'}{P \boxplus Q \xrightarrow{\mu} P'} \quad \frac{P \xrightarrow{\mu} P' \quad Q \xrightarrow{\mu} Q'}{P \boxplus Q \xrightarrow{\mu} P' \boxplus Q'} \quad \frac{P \xrightarrow{a} P' \quad Q \xrightarrow{a} Q'}{P \boxplus Q \xrightarrow{a} P' \boxplus Q'}$$

$$\begin{array}{ccc} \mathcal{T}X & \xrightarrow{\llbracket \cdot \rrbracket_\beta} & \Omega \\ \downarrow \beta & & \downarrow \\ \mathcal{F}\mathcal{T}X & \xrightarrow{\mathcal{F}(\llbracket \cdot \rrbracket_\beta)} & \mathcal{F}(\Omega) \end{array}$$

Final $\mathcal{F}$ -coalgebra:
semantic universe,
space of all behaviours

There exists a
distributive law

$$\lambda : \mathcal{T}\mathcal{F} \rightarrow \mathcal{F}\mathcal{T}$$

Diagram lives in the
category $\mathcal{T}\text{-Alg}$

$\llbracket \cdot \rrbracket_\beta$ is compositional, e.g.

$$\llbracket s + t \rrbracket = \llbracket s \rrbracket + \llbracket t \rrbracket$$

Structural Operational Semantics: the String Diagram case

Goals

A principled approach to
the SOS of string diagrams,
in the style of the Turi-Plotkin framework.

String diagram **syntax** as **monads**

$\mathcal{T}X$

String diagram **behaviour** as **coalgebras**

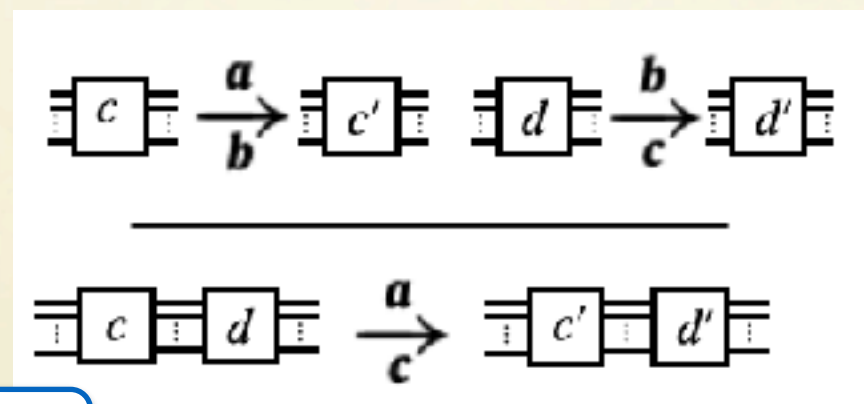
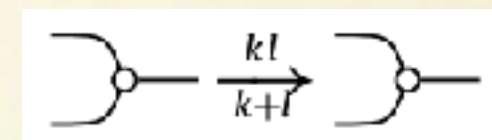
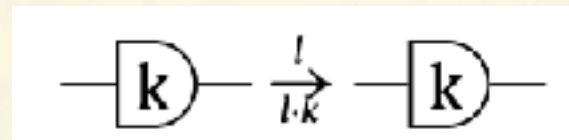
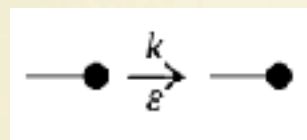
$\mathcal{T}X \xrightarrow{\beta} \mathcal{F}\mathcal{T}X$

Compositional semantics via **distributive laws**

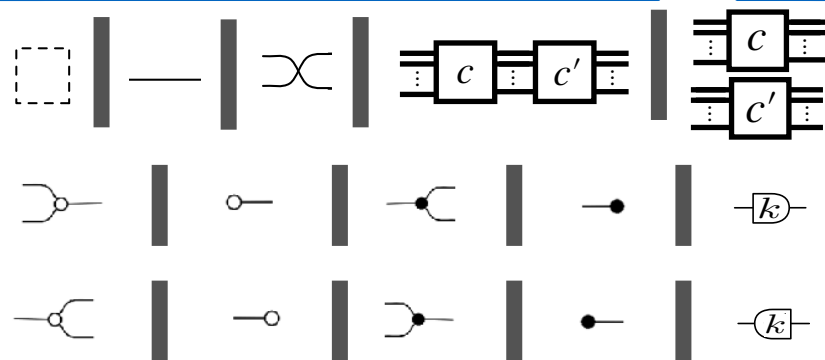
$\lambda : \mathcal{T}\mathcal{F} \rightarrow \mathcal{F}\mathcal{T} \qquad \mathcal{T}X \xrightarrow{\llbracket \cdot \rrbracket_{\beta}} \Omega$

SOS Rules for String Diagrams

Example: Signal Flow Diagrams



...



Which of these choices are **principled**?

How can we ensure bisimilarity is a **congruence**?

The Categorical Picture: Syntax

Category of signatures

$$\text{Sig} := \text{Span}(\text{Set})(\mathbb{N}, \mathbb{N})$$

$a(o)$ = arity of o

$$\mathbb{N} \xleftarrow{a} \Sigma \xrightarrow{c} \mathbb{N}$$

$c(o)$ = coarity of o

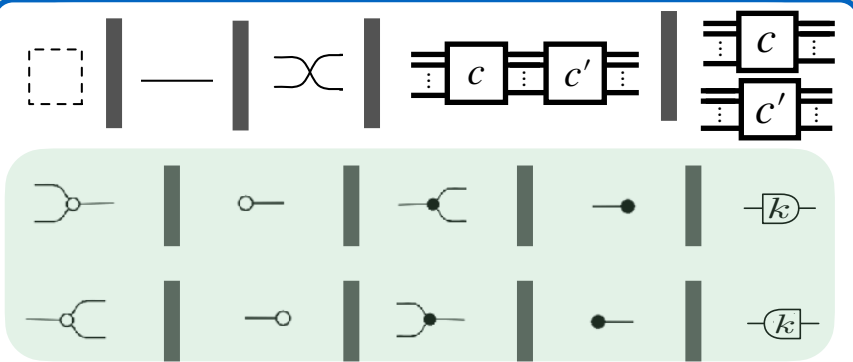
Set of
generators

Example: Signal Flow Diagrams

$$\mathbb{N} \xleftarrow{a} \Sigma \xrightarrow{c} \mathbb{N}$$

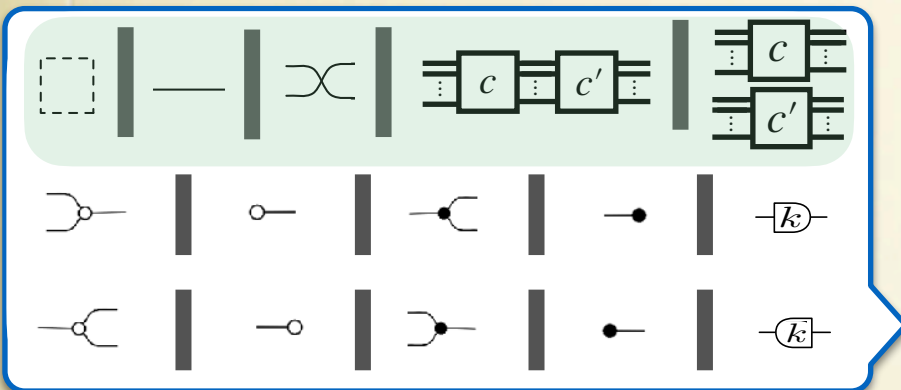
$$a(\text{---}\bullet\text{---}) = 2$$

$$c(\text{---}\bullet\text{---}) = 1$$



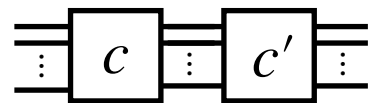
From Generators to String Diagrams

We define a composite endofunctor $\mathbf{Sig} \rightarrow \mathbf{Sig}$



$$S := (Id; Id) + \iota + (Id \oplus Id) + \epsilon + \sigma$$

sequential composition

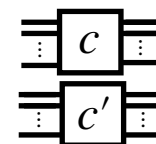


(defined by pullback)

constant



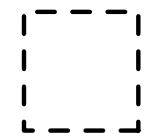
parallel composition



constant



constant



S yields a free monad $S^\dagger : \mathbf{Sig} \rightarrow \mathbf{Sig}$

Quotient
of $S^\dagger$ by
the laws
of SMCs

$S(\Sigma)$

One-step Σ -string diagrams

$S^\dagger(\Sigma)$

Σ -string diagrams

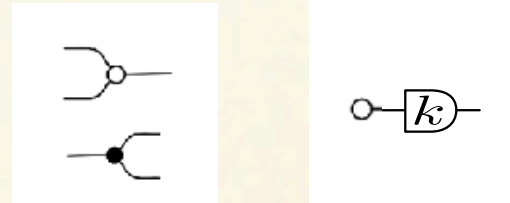
$S^\dagger_{\text{SM}}(\Sigma)$

Σ -string diagrams quotiented
by the laws of SMCs

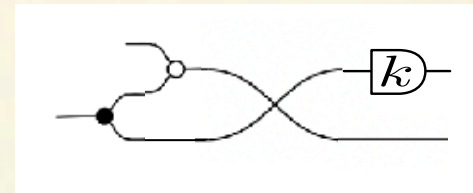
From Generators to String Diagrams

Example: Signal Flow Diagrams

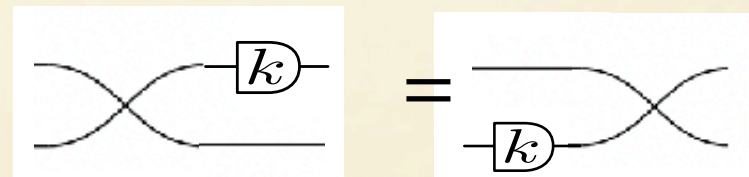
$S(\Sigma)$



$S^\dagger(\Sigma)$



$S_{SM}^\dagger(\Sigma)$



Quotient
of $S^\dagger$ by
the laws
of SMCs

$S(\Sigma)$

One-step Σ -string diagrams

$S^\dagger(\Sigma)$

Σ -string diagrams

$S_{SM}^\dagger(\Sigma)$

Σ -string diagrams quotiented
by the laws of SMCs

The Categorical Picture: Semantics

$$\mathcal{F} := \overline{\mathcal{P}}_{\kappa}(L; Id; L) : \mathbf{Sig} \rightarrow \mathbf{Sig}$$

Powerset functor on **Sig**

(constant) label functor
mapping any Σ into

$$\mathbb{N} \xleftarrow{|\cdot|} A^* \xrightarrow{|\cdot|} \mathbb{N}$$

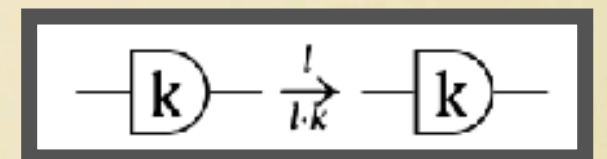
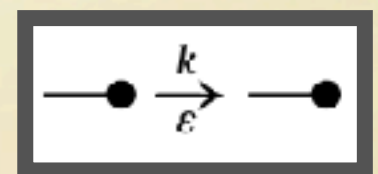
Cf. 'classic' LTS:

$$F = P(A \times Id)$$

A coalgebra $\beta : \Sigma \rightarrow F(\Sigma)$ defines **transition rules**.

Example: Signal Flow Diagrams

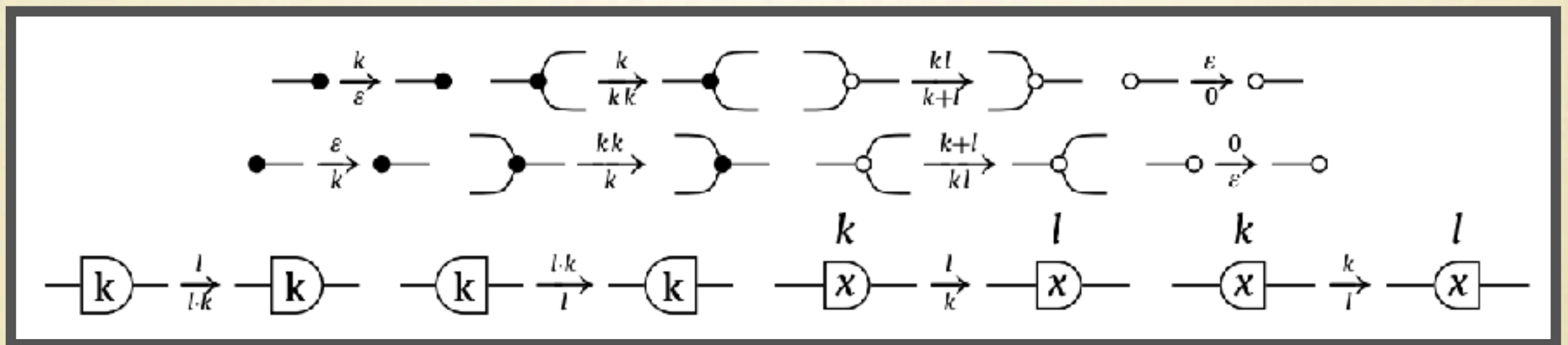
$$\beta : \begin{array}{ll} \text{---} \bullet & \mapsto \{ (k, \text{---} \bullet, \epsilon) \mid k \in \mathbb{R} \} \\ \text{---} \boxed{k} \text{---} & \mapsto \{ (l, \text{---} \boxed{k} \text{---}, lk) \mid l \in \mathbb{R} \} \end{array}$$



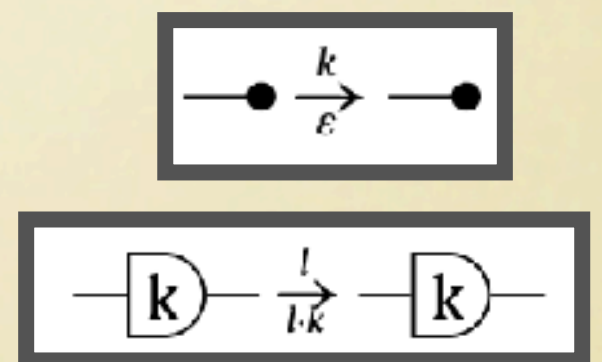
The Categorical Picture: Semantics

Example: Signal Flow Diagrams

Full definition of β as transition rules



$$\beta: \begin{array}{ll} \text{---} \bullet & \mapsto \{ (k, \text{---} \bullet, \epsilon) \mid k \in \mathbb{R} \} \\ \text{---} \boxed{k} & \mapsto \{ (l, \text{---} \boxed{k}, lk) \mid l \in \mathbb{R} \} \end{array}$$



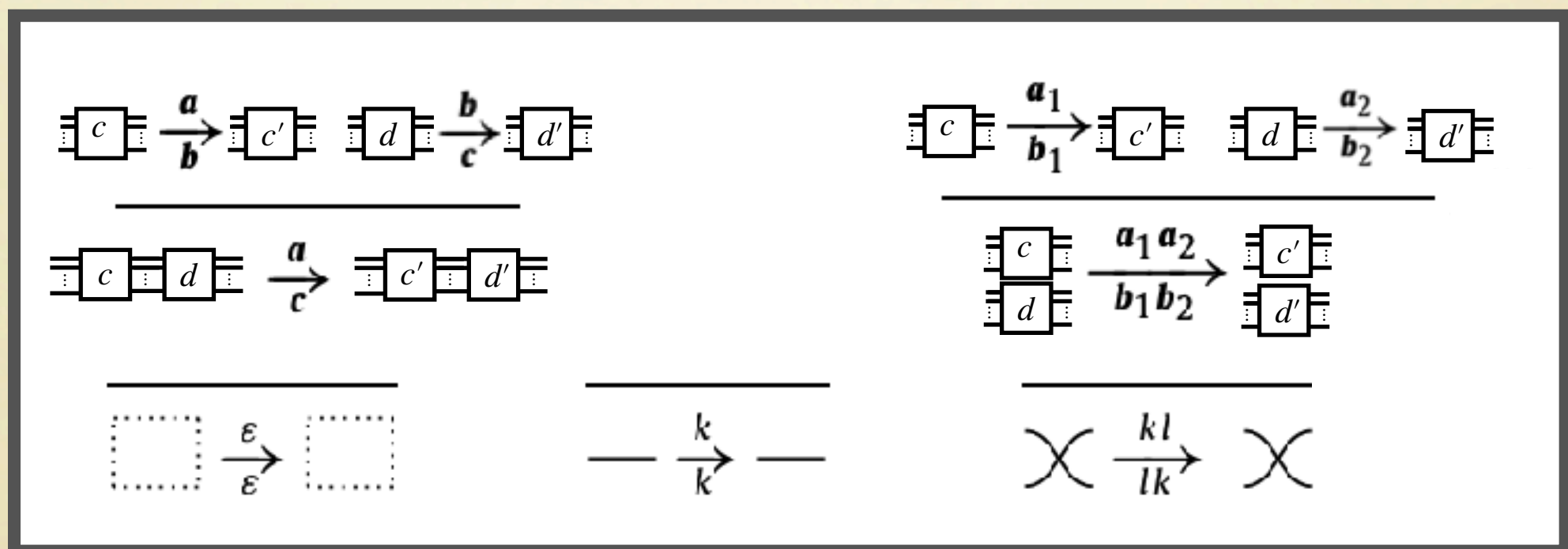
The Categorical Picture: Distributive Law

Theorem

There is a distributive law

$$\lambda: S_{SM}^\dagger F \Rightarrow F S_{SM}^\dagger$$

$\lambda_\Sigma: S_{SM}^\dagger(F(\Sigma)) \rightarrow F(S_{SM}^\dagger(\Sigma))$ is defined by these transition rules:



The Payoff: Compositionality

$$\begin{array}{ccc}
 S_{SM}^\dagger(\Sigma) & \xrightarrow{\llbracket \cdot \rrbracket_\beta} & \Omega \\
 \downarrow \beta^\sharp & & \downarrow \\
 \mathcal{F}(S_{SM}^\dagger(\Sigma)) & \xrightarrow{\mathcal{F}(\llbracket \cdot \rrbracket_\beta)} & \mathcal{F}(\Omega)
 \end{array}$$

$\llbracket \equiv c \equiv \rrbracket_\beta$ = labeled transition systems from $\equiv c \equiv$ modulo bisimilarity

$\llbracket \equiv c \equiv \rrbracket_\beta$ is both a F -coalgebra and $S_{SM}^\dagger$ -algebra map:

bisimilarity is a **congruence** with respect to the SMC structure.

Further Developments I Beyond Symmetric Monoidal

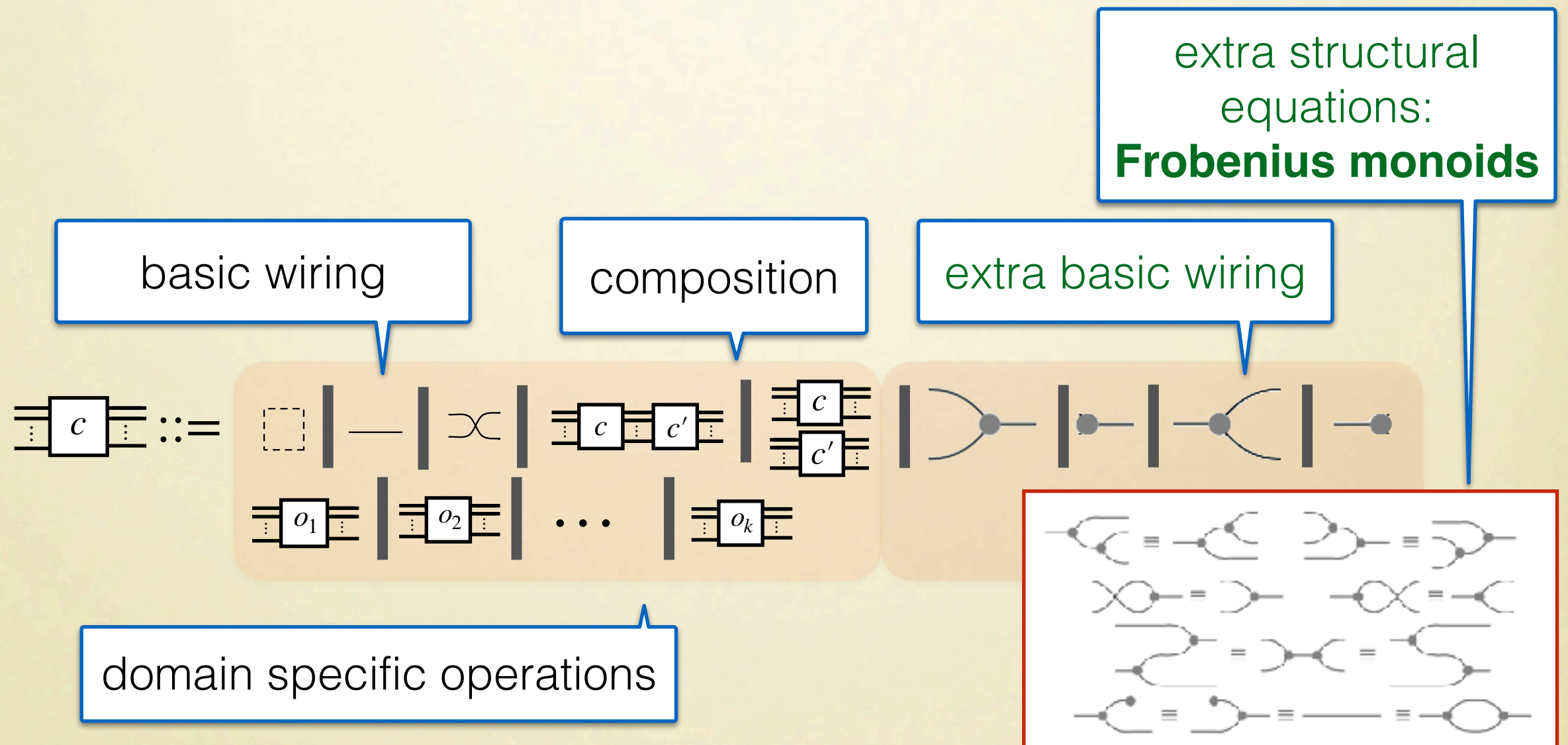
Beyond Symmetric Monoidal

The same developments apply to
richer categorical structures...

...with some notable exception!

Hypergraph Categories

Example: Hypergraph Categories



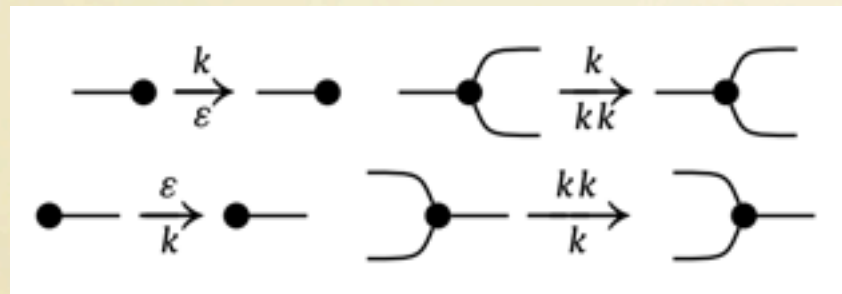
Hypergraph Categories

Theorem There are **two** distributive laws of type

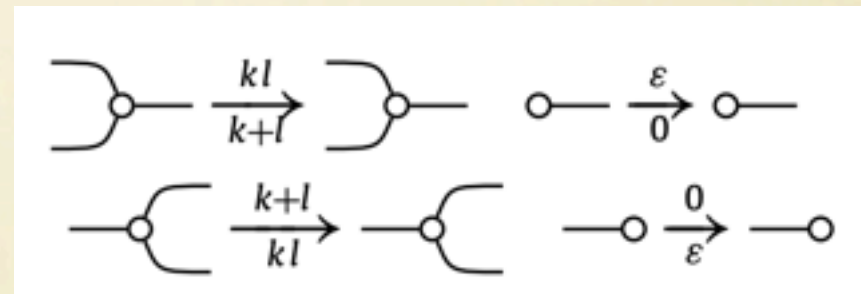
$$\lambda: S_{Hyp}^{\dagger} F \Rightarrow F S_{Hyp}^{\dagger}$$

Assuming the set of labels is an abelian group

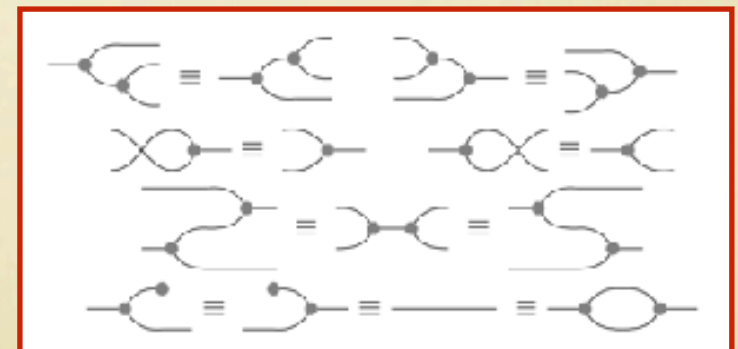
`black' law
presented by rules



`white' law
presented by rules

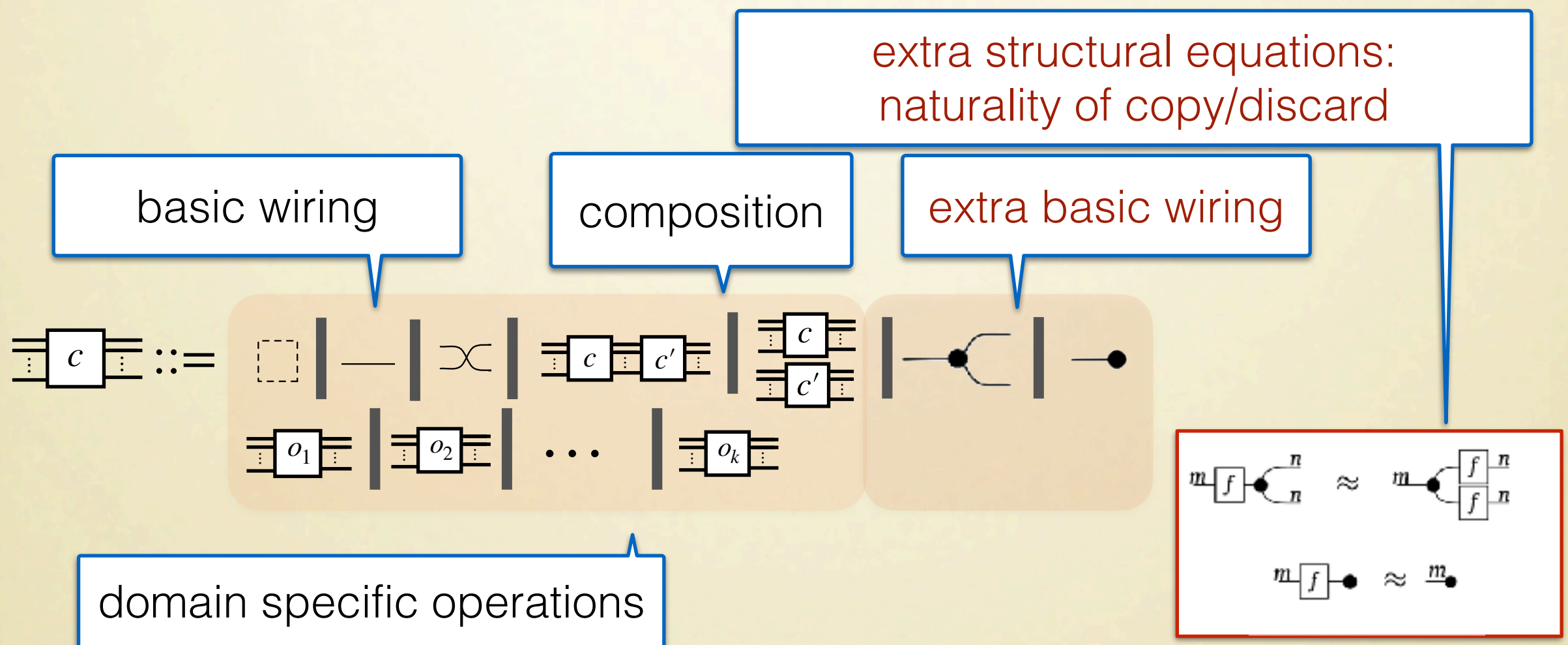


Compositionality of the resulting semantics means that it is compatible with the Frobenius equations



Cartesian Categories

Non-example: Cartesian categories (Lawvere Theories)



Cartesian Categories

Non-example: Cartesian categories (Lawvere Theories)

Counterexample: SOS specification for a string diagrammatic language over a cartesian category

$$\beta: \Sigma \rightarrow F(\Sigma)$$

$$\text{---}\bigcirc\text{---} \xrightarrow[q]{\epsilon} \text{---}\bigcirc\text{---}$$

$$\text{---}\bigcirc\text{---} \xrightarrow[b]{\epsilon} \text{---}\bigcirc\text{---}$$

A distributive law of the required type $\lambda: S_{Cart}^\dagger F \Rightarrow FS_{Cart}^\dagger$ forces:

$$\begin{array}{c} \boxed{d} \text{---} \xrightarrow[b]{\varepsilon} \boxed{d} \text{---} \quad \text{---} \bullet \xrightarrow[b]{b} \text{---} \bullet \text{---} \\ \hline \boxed{d} \text{---} \bullet \xrightarrow[b]{\varepsilon} \boxed{d} \text{---} \bullet \end{array}$$

$\begin{array}{c} \boxed{d} \text{---} \xrightarrow[a]{\varepsilon} \boxed{d} \text{---} \quad \boxed{d} \text{---} \xrightarrow[b]{\varepsilon} \boxed{d} \text{---} \\ \hline \begin{array}{c} \boxed{d} \text{---} \\ \boxed{d} \text{---} \end{array} \xrightarrow[ab]{\varepsilon} \begin{array}{c} \boxed{d} \text{---} \\ \boxed{d} \text{---} \end{array} \end{array}$

$$\begin{array}{ccc} \boxed{d} \text{---} & \xrightarrow[b]{\varepsilon} & \boxed{d} \text{---} \end{array} \quad \begin{array}{ccc} \boxed{d} \text{---} & \xrightarrow[a]{\varepsilon} & \boxed{d} \text{---} \end{array}$$

$$\begin{array}{ccc} \boxed{d} \text{---} & \xrightarrow[b \bar{a}]{\varepsilon} & \boxed{d} \text{---} \end{array}$$

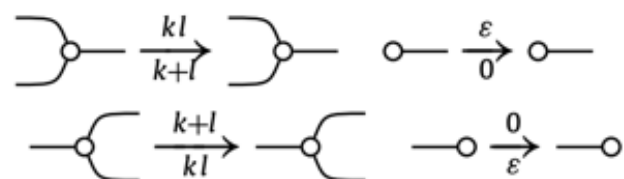
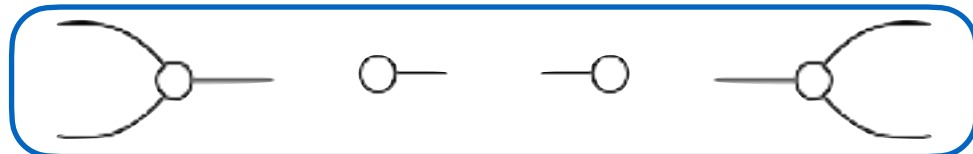
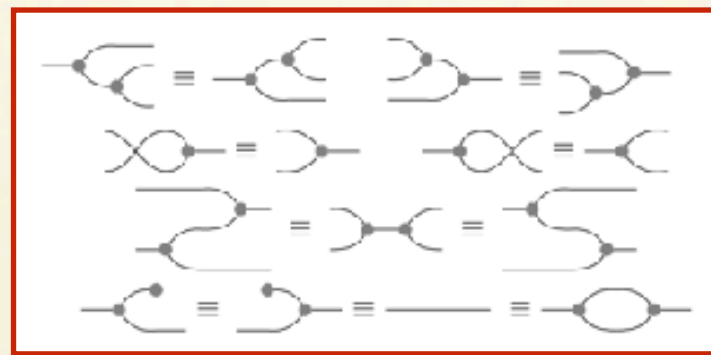
violates

Further Developments II

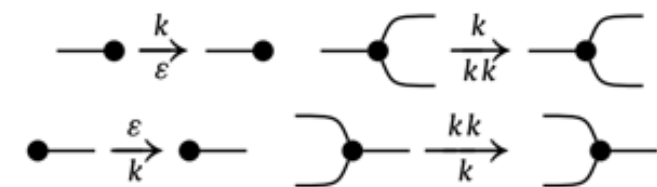
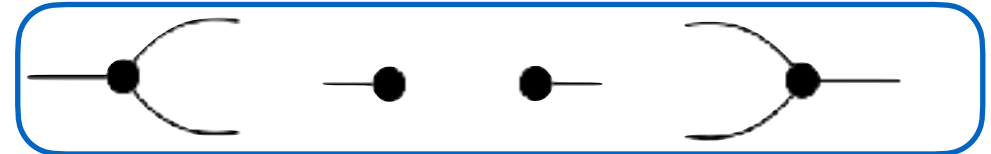
Synchronisation: Milner vs Hoare

Synchronisation: Hoare vs Milner

We interpret process calculi as string diagrammatic languages with Frobenius structure



`White Frobenius' semantics:
synchronisation à la Milner (SCCS)



`Black Frobenius' semantics:
synchronisation à la Hoare (CSP)

A Simple Process Calculus

A simple process calculus syntax:

Parallel composition

Process variables

$$P ::= P \mid P \mid \nu a_1(P) \mid f \mid P\sigma$$

Finitely-supported
permutation on names*

Name restriction/hiding

*Technicality that connects symbolic (variables) and graphical (wires) syntax.
(More elegant alternative: nominal string diagrams)

Semantics: Milner vs Hoare

Actions $\alpha, \beta, \dots$ are maps $\mathcal{N} \rightarrow M$ from a set $\mathcal{N}$ of names to a monoid M .

$$P \xrightarrow{\alpha} Q$$

Process declarations

$$\frac{}{f \xrightarrow{0} f} \quad \frac{f := \sum_{i \in I} \alpha_i . P_i}{f \xrightarrow{\alpha_i} P_i} \quad \frac{P \xrightarrow{\alpha} P'}{P\sigma \xrightarrow{\alpha \circ \sigma} P'\sigma}$$

Hoare Synchronisation

$$M = (2, \cup, 0)$$

$$\frac{P \xrightarrow{\alpha} P' \quad Q \xrightarrow{\beta} Q' \quad \alpha \cap al(Q) = \beta \cap al(P)}{P|Q \xrightarrow{\alpha \cup \beta} P'|Q'}$$

$$\frac{P \xrightarrow{\alpha} P'}{va_i(P) \xrightarrow{\alpha \setminus \{a_i\}} va_i(P')}$$

Milner Synchronisation

$$M = (\mathbb{Z}, +, 0)$$

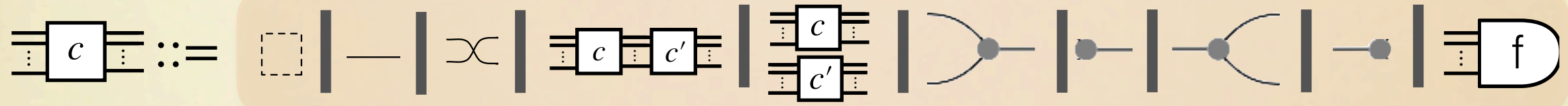
$$\frac{P \xrightarrow{\alpha} P' \quad Q \xrightarrow{\beta} Q'}{P|Q \xrightarrow{\alpha + \beta} P'|Q'}$$

$$\frac{P \xrightarrow{\alpha} P'}{va_i(P) \xrightarrow{\alpha} va_i(P')} \quad \alpha(a_i) = 0$$

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau} P'|Q'}$$

String Diagrammatic Interpretation

$$f \in \mathcal{V}$$



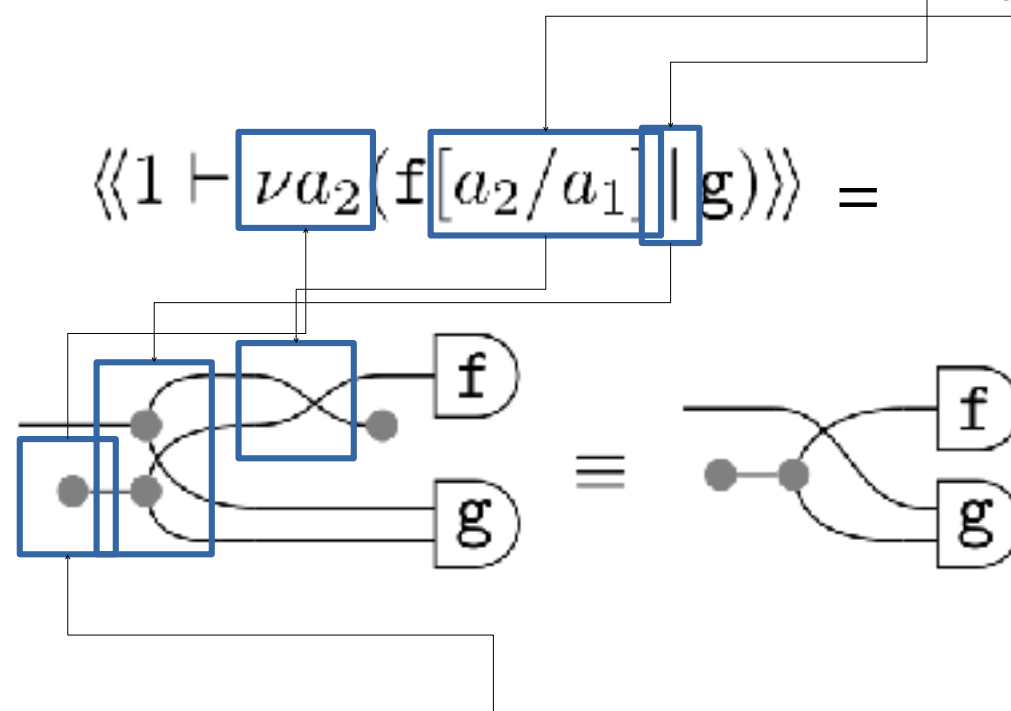
$$\langle\langle n \vdash \mathbf{f} \rangle\rangle = \text{---} \boxed{\mathbf{f}}$$

$$\langle\langle n + 1 \vdash P \rangle\rangle = \text{---} \bullet \boxed{\langle\langle P \rangle\rangle} \qquad \langle\langle n \vdash P|Q \rangle\rangle = \text{---} \bullet \begin{array}{c} \boxed{\langle\langle P \rangle\rangle} \\ \boxed{\langle\langle Q \rangle\rangle} \end{array}$$

$$\langle\langle n \vdash \nu a_{n+1}(P) \rangle\rangle = \text{---} \bullet \boxed{\langle\langle P \rangle\rangle} \qquad \langle\langle n \vdash P\sigma \rangle\rangle = \text{---} \boxed{\bar{\sigma}} \text{---} \bullet \boxed{\langle\langle P \rangle\rangle}$$

Example

Permutation swapping the two variables
Parallel composition



Restriction/hiding

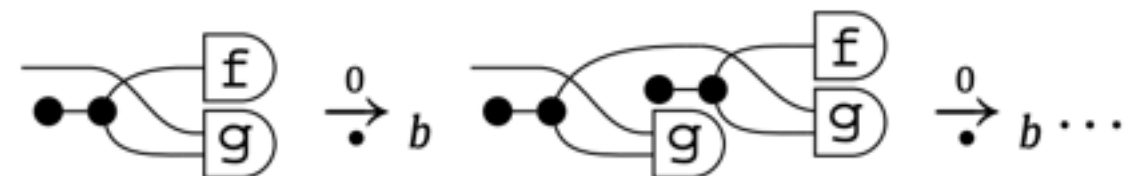
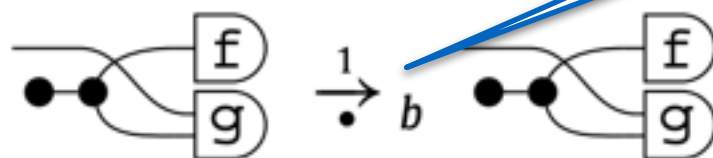
'Black Frobenius' semantics
= Hoare Synchronisation

$$\frac{P \xrightarrow{\alpha} P' \quad Q \xrightarrow{\beta} Q' \quad \alpha \cap al(Q) = \beta \cap al(P)}{P|Q \xrightarrow{\alpha \cup \beta} P'|Q'}$$

$$\frac{P \xrightarrow{\alpha} P'}{\nu a_i(P) \xrightarrow{\alpha \setminus \{a_i\}} \nu a_i(P')}$$

$f := a_1. \nu a_2 (f[a_2/a_1] \mid g)$

$g := a_1.g + a_2.g$



$1 := \alpha : a_1 \mapsto 1 \quad 0 := \alpha : a_1 \mapsto 0$

Hoare, Milner, Frobenius

Theorem

Hoare-style SOS

$$P \xrightarrow{\alpha}_H P'$$

Milner-style SOS

$$P \xrightarrow{\alpha}_M P'$$

Restriction of α to $\{a_1, \dots, a_n\}$

$$n \text{ --- } \langle\langle P \rangle\rangle \xrightarrow[n \vdash \alpha]{\bullet} b \text{ --- } \langle\langle P' \rangle\rangle$$

$$n \text{ --- } \langle\langle P \rangle\rangle \xrightarrow[n \vdash \alpha]{\bullet} w \text{ --- } \langle\langle P' \rangle\rangle$$

Black Frobenius λ

$$\begin{array}{cc} \xrightarrow{\bullet} \xrightarrow[k]{\varepsilon} \xrightarrow{\bullet} & \xrightarrow{\bullet} \xrightarrow[kk]{k} \xrightarrow{\bullet} \\ \bullet \xrightarrow[k]{\varepsilon} \bullet & \xrightarrow{\bullet} \xrightarrow[k]{kk} \xrightarrow{\bullet} \end{array}$$

White Frobenius λ

$$\begin{array}{cc} \xrightarrow{\circ} \xrightarrow[k+l]{kl} \xrightarrow{\circ} & \xrightarrow{\circ} \xrightarrow[0]{\varepsilon} \xrightarrow{\circ} \\ \xrightarrow{\circ} \xrightarrow[kl]{k+l} \xrightarrow{\circ} & \xrightarrow{\circ} \xrightarrow[\varepsilon]{0} \xrightarrow{\circ} \end{array}$$

Conclusions

Wrap Up

- A general framework for giving SOS to string diagrammatic languages
- Principled approach following the Turi-Plotkin framework
- Study of suitable distributive laws for each monoidal structure
- Frobenius interpretation of different synchronisation mechanisms in process algebra
- What's next? SOS for probabilistic models (e.g. Bayesian networks)

Want to know more?

- F. Bonchi, R. Piedeleu, P. Sobocinski, F.Z.
Bialgebraic Foundations for the Operational Semantics of String Diagrams
<https://www.sciencedirect.com/science/article/abs/pii/S0890540121000821>
- R. Piedeleu, F.Z.
An Introduction to String Diagrams for Computer Scientists
<https://www.cambridge.org/core/elements/an-introduction-to-string-diagrams-for-computer-scientists/3CDAF8F57D2299F0EACA3354E9757CFD>
- F. Bonchi, P. Sobocinski, F.Z.
A Survey on Compositional Signal Flow Theory
<http://www.zanasi.com/fabio/files/paperIFIP21.pdf>